

Practical: 5

Implementation of a knapsack problem using dynamic programming.

/08/2026

Aim: Implementation of a knapsack problem using dynamic programming.

Algorithm:

Algorithm:-

Input:- number of items, their weights and profit and the maximum capacity of knapsack

Output:- Maximum profit that can be obtained without exceeding the knapsack capacity

Step 1:- Start

Step 2:- Read the number of items n .

Step 3:- Read the weight $w[i]$ and profit $w[i]$ of each item.

Step 4:- Create a DP table $k[n+1][w+1]$

Step 5:- initialize the first row and first column of the DP table with 0

Step 6:- for each item $i=1$ to n , and for each capacity $j=1$ to w ;
if $w[i-1] \leq j$, calculate
 $k[i][j] = \max(v[i-1] + k[i-1][j-w[i-1]], k[i-1][j])$
otherwise
 $k[i][j] = k[i-1][j]$

Step 7:- Repeat the process until all items and capacities are considered

Step 8:- The maximum profit is stored in $k[n][w]$

Step 9:- Display the maximum profit

Step 10:- Stop

Program:

```
#include<iostream>
using namespace std;

class Knapsack{
    private:

        int w[10];
        int v[10];
        int n;
        int W;
        int dp[10][100];

    public:
        void getInput() {
            cout << "Enter number of items  : ";
            cin >> n;
            cout << "Enter knapsack capacity : ";
            cin >> W;
            for(int i = 1; i <= n; i++) {
                cout << "Item " << i << " weight and value : ";
                cin >> w[i] >> v[i];
            }

        }

        void solve() {

            for(int i = 0; i <= n; i++) dp[i][0] = 0;
            for(int j = 0; j <= W; j++) dp[0][j] = 0;

            for(int i = 1; i <= n; i++) {
                for(int j = 1; j <= W; j++) {
```

```
        if(w[i] <= j) {  
            dp[i][j] = max(  
                v[i] + dp[i-1][j-w[i]],  
                dp[i-1][j]  
            );  
        } else {  
            dp[i][j] = dp[i-1][j];  
        }  
    }  
}
```

```
void printTable() {  
    cout << "\nDP Table:" << endl;  
    cout << "    ";  
    for(int j = 0; j <= W; j++)  
        cout << j << " ";  
    cout << endl;
```

```
    for(int i = 0; i <= n; i++) {  
        cout << "i=" << i << " ";  
        for(int j = 0; j <= W; j++)  
            cout << dp[i][j] << " ";  
        cout << endl;  
    }  
}
```

```
void findItems() {  
    cout << "\nSelected Items:" << endl;  
    int j = W;  
    for(int i = n; i >= 1; i--) {  
        if(dp[i][j] != dp[i-1][j]) {  
            cout << "Item " << i  
                << " (w=" << w[i]  
                << ", v=" << v[i]  
                << ") TAKEN " << endl;  
            j = j - w[i];  
        }  
    }
```

```
        } else {
            cout << "Item " << i
                << " (w=" << w[i]
                << ", v=" << v[i]
                << ") NOT taken " << endl;
        }
    }
}
```

```
void showResult() {
    cout << "\nMaximum Value = "
        << dp[n][W] << endl;
}
};
```

```
int main() {

    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}
```

Output:

```
C:\Users\Admin\OneDrive\Pic × + v
Enter number of items : 4
Enter knapsack capacity : 8
Item 1 weight and value : 3
2
Item 2 weight and value : 4
3
Item 3 weight and value : 5
4
Item 4 weight and value : 6
1

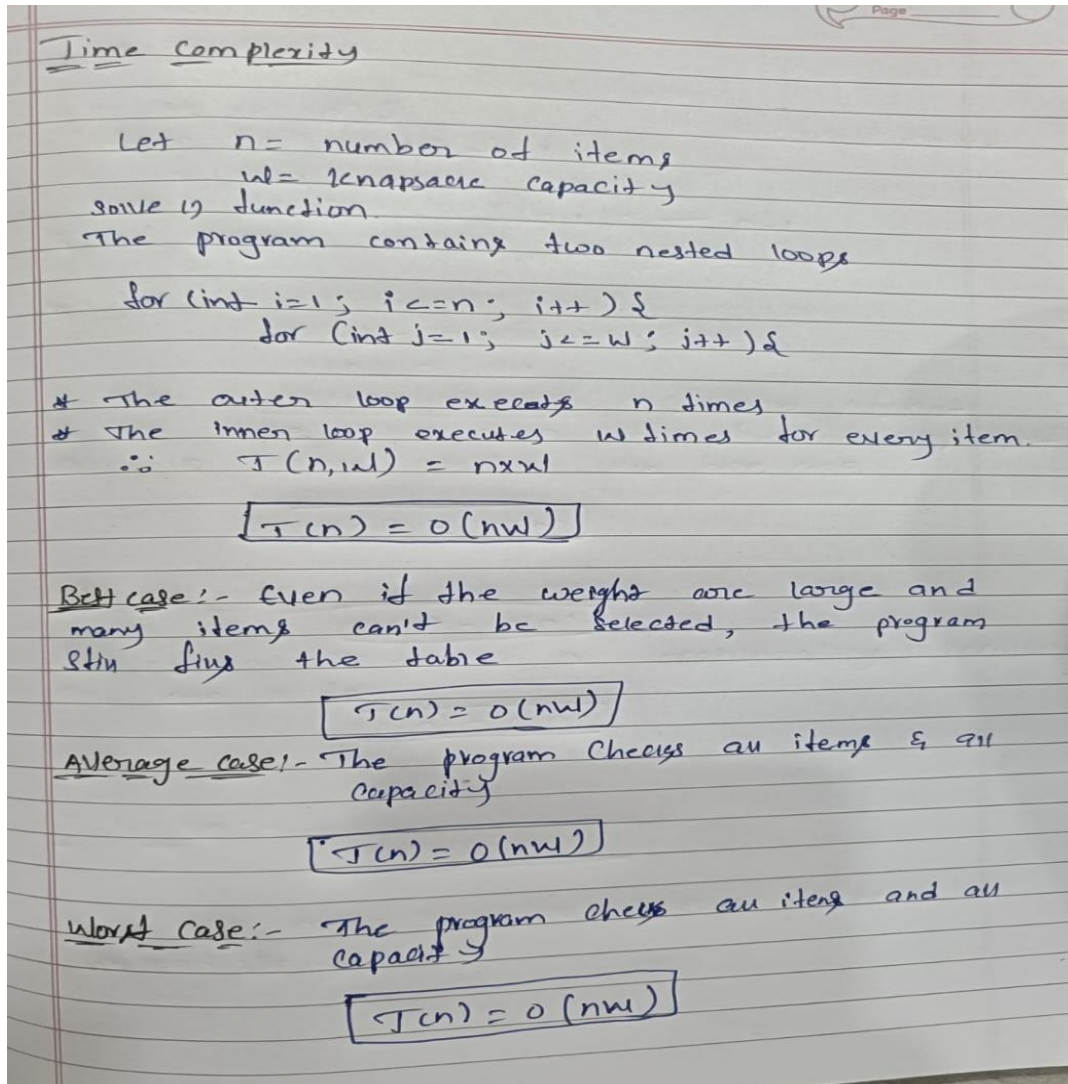
DP Table:
    0  1  2  3  4  5  6  7  8
i=0  0  0  0  0  0  0  0  0  0
i=1  0  0  0  2  2  2  2  2  2
i=2  0  0  0  2  3  3  3  5  5
i=3  0  0  0  2  3  4  4  5  6
i=4  0  0  0  2  3  4  4  5  6

Maximum Value = 6

Selected Items:
Item 4 (w=6, v=1) NOT taken
Item 3 (w=5, v=4) TAKEN
Item 2 (w=4, v=3) NOT taken
Item 1 (w=3, v=2) TAKEN

-----
Process exited after 24.35 seconds with return value 0
Press any key to continue . . .
```

Time Complexity:



Time Complexity

Let n = number of items
 w = knapsack capacity
solve is function.

The program contains two nested loops

```
for (int i=1; i<=n; i++) {  
    for (int j=1; j<=w; j++) {
```

* The outer loop executes n times
* The inner loop executes w times for every item.
 $\therefore T(n, w) = n \times w$

$T(n) = O(nw)$

Best case:- Even if the weights are large and many items can't be selected, the program still finds the table

$T(n) = O(nw)$

Average case:- The program checks all items & all capacity

$T(n) = O(nw)$

Worst case:- The program checks all items and all capacity

$T(n) = O(nw)$

Conclusion:

The 0/1 Knapsack Problem was successfully implemented using Dynamic Programming. The DP approach efficiently finds the maximum possible value without exceeding the knapsack capacity by storing and reusing solutions to smaller subproblems.